

Relatório G15

Cláudio Ferreira^[108577], Marco Abreu^[108578] e Nelson Sousa^[109068]

Universidade Do Minho, Braga, Portugal
gcii@reitoria.uminho.pt

1. Introdução

Este relatório tem como objetivo explicar o funcionamento dos códigos do cliente e do servidor que compõem o nosso projeto, cujo propósito é implementar um serviço que permita a indexação e pesquisa de documentos de texto armazenados localmente num computador.

2. Programa Cliente

O programa cliente é composto por duas funções:

2.1. print_usage

A função print_usage tem como único objetivo escrever no ecrã, através da função write, a forma de utilização do programa cliente, sendo apenas invocada quando o input do programa está incorreto.

```
nelson-sousa@nelson-sousa-Legion-5-15ACH6H:~/Desktop/ProjetoS0$ ./dclient
Modos de utilização:

dclient -a <Titulo> <Autores> <Ano> <Caminho>
dclient -c <Chave>
dclient -d <Chave>
dclient -l <Linha> <Palavra>
dclient -s <Palavra>
dclient -s <Palavra> <NºProcessos>
dclient -f
```

2.2. Main

A função main tem como objetivo ler os inputs do cliente, neste caso os comandos que por ele são introduzidos, e enviar para o servidor.

Começa então por verificar se o input do cliente possui mais que dois argumentos, uma vez que não existe nenhum input com menos de dois argumentos.

Após esta breve verificação, o programa cria um FIFO para o cliente atual em específico, nomeando-o através do seu pid, e atribuindo as permissões de leitura e escrita através de `umask(0)`, criando o fifo a partir da função `mkfifo`.

Seguindo o código, é criado um buffer e uma string onde será armazenada a operação pedida pelo cliente, verificando qual é a operação através dos ifs e quando descobre-a armazena a instrução no buf, formatada para que a leitura feita pelo servidor seja facilitada. A função também verifica que no caso de o input ter sido incorreto, mas a função não tenha terminado antes, recorra outra vez á função `print_usage` e dá unlink ao fifo anteriormente criado.

Nesta seguinte etapa, o programa cliente verifica se o servidor está a correr e, no caso de não estar, a função imprime no ecrã a mensagem “Aguardando o servidor ser iniciado...” e recorre á função `usleep` para colocar o código em espera até tentar outra vez, tendo o nosso grupo selecionado o valor de 500ms de espera entre cada verificação.

A seguir é aberto o FIFO do servidor, verificando se o FIFO foi aberto corretamente através de um if, é também verificado se o buffer tem uma mensagem maior que 512 bytes, uma vez que o enunciado do trabalho pede para assumir que o tamanho total dos argumentos não é maior que 512 bytes. Após estas duas verificações, envia então o buffer para o FIFO do servidor.

Por último, o programa abre o FIFO do cliente, onde o servidor armazenou a resposta, e dá `printf` da resposta, garantindo que a string da resposta seja terminada. Antes de encerrar o programa, é fechado o file descriptor (`fd`) e o FIFO do cliente.

3. Programa Servidor

O programa servidor começa por definir uma estrutura denominada “Doc”, para representar cada documentado armazenado no servidor, sendo estes documentos armazenados através de uma lista ligada.

```
typedef struct Doc {
    int id;
    char title[256];
    char authors[256];
    int year;
    char path[256];
    struct Doc* next;
} Doc;
```

De seguida são definidas as variáveis globais, `head`, que armazena a cabeça da lista de documentos, `next_id` que armazena o próximo id a ser atribuído, `base_folder` que representa a pasta de documentos recebida no input do servidor, `cache_size` que limita o número de documentos adicionados na lista, sendo um dos argumentos recebidos no input do servidor, `running` que serve para manter o servidor ativo e por último a variável `documentos_adicionados_na_sessao` serve como ferramenta de comparação com `cache_size`, para que o número de documentos adicionados não ultrapasse `cache_size`.

```
Doc* head = NULL;
int next_id = 1;
char base_folder[256];
int cache_size = 0;
int running = 1;
int documentos_adicionados_na_sessao = 0;
```

Após as variáveis globais é definida outra estrutura, mais simples, que é apenas utilizada na função pesquisaDocumentos, e serve para que na pesquisa paralela seja mantida uma associação entre o PID de um processo filho e o ID do documento que está a ser processado. Esta estrutura foi apenas criada pois, ao retornar a resposta ao cliente, no caso de o input ser “./dclient -s...”, ocorria um erro no output dependendo do número de processos que eram usados, tendo corrigido esse erro.

```
typedef struct {
    pid_t pid;
    int doc_id;
} ProcessInfo;
```

3.1. load_index e save_index

As funções load_index e save_index tem como propósito manter a database do servidor mesmo que este seja encerrado, armazenando a informação sobre os documentos anteriormente armazenados em index.db.

O ficheiro index.db armazena todos os documentos do servidor da forma id|title|authors|year|path, para que depois sejam lidos pela função load_index e possa ser iterada pelo programa

A função load_index abre o ficheiro index.db e lê o documento formatando-o para um formato de lista ligada, para que possa ser iterada pelo programa. Esta função itera pelo documento sabendo que cada linha representa um documento diferente.

Já a função save_index serve para guardar as informações sobre os documentos do servidor dentro de index.db, armazenando os novos documentos adicionados na database do servidor.

3.2. Resposta

A função respostas serve apenas para escrever em cliente_fifo a resposta que será emitida pelo servidor após a execução do pedido feito pelo cliente.

3.3. adicionarDocumento

A função adicionarDocumento serve para adicionar os novos documentos ao servidor verificando antes de adicionar se já foi ultrapassado o cache_size, se o caminho do documento existe e se há memória suficiente para o armazenamento deste.

Após estas verificações é então adicionado o novo documento á lista ligada.

3.4. consultarDocumento

A função consultarDocumento recebe uma chave que corresponde ao identificador do documento a depois atravessa a lista ligada até encontrar o identificador, ao encontrar armazena no buf a informação sobre este documento e envia para a função resposta. No caso de o identificador não pertencer a nenhum documento é enviado para a função resposta a string “Chave não existe”.

3.5. removerDocumento

A função `removerDocumento` percorre a lista ligada de documentos e ao atingir o índice pretendido para a remoção, retira o documento da lista e dá `free`, para libertar a memória antes ocupada pelo mesmo. No caso de o identificador não pertencer a nenhum documento é enviado para a função resposta a string “Chave não existe”.

3.6. contaLinhasDocumento

A função `contaLinhasDocumento` percorre a lista ligada de documentos, e ao encontrar o documento que tem a chave procurada, percorre-o através de processos filhos, em que este recorre ao uso de um pipe para comunicar com o processo pai. Após percorrer o documento armazena em `countbuf` o número de linhas e manda para a função resposta.

3.7. pesquisaDocumentos

A função `pesquisaDocumentos` implementa o uso de busca paralela em múltiplos documentos, utilizando processos independentes para maximizar o desempenho. A função recorre a um array `ids` para armazenar os identificadores dos documentos e utiliza a estrutura `active_processes`, anteriormente mencionada, para gerir os seus processos e os respetivos documentos. Após usar processos filho para procurar nos respetivos documentos através da função `grep`, armazena os `ids` no array e, para garantir que no output está presente uma lista ordenada de `ids` de documentos realiza um `qsort`, com a condição definida em `compare_ids`, tal que após este `qsort` a função envia o array final à função resposta. Após o envio do array a função liberta toda a memória gasta com o recurso à função `free`.

3.8. encerrarServidor

A função `servidor` tem apenas como objetivo mudar o valor da variável `running`, uma vez que o propósito desta variável é informar se o servidor deve estar ligado ou não.

3.9. Main

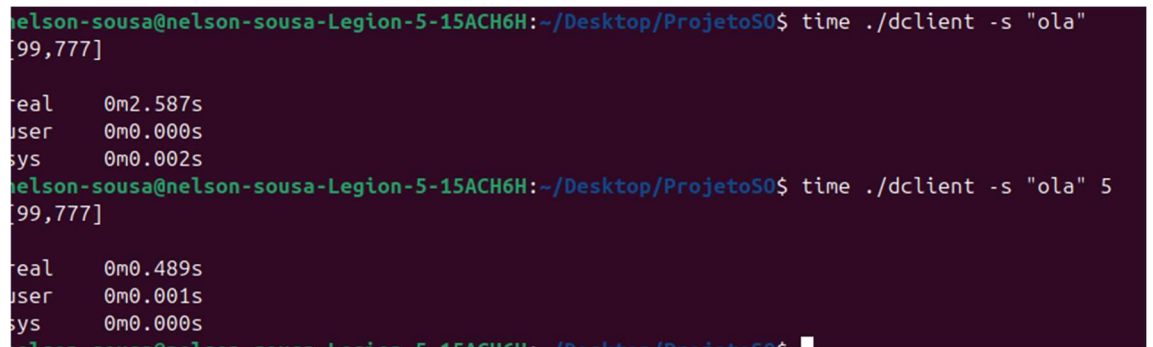
A função `main` começa por verificar se o input, ao inicializar o servidor, possui três argumentos. Após esta verificação é copiado o nome do ficheiro onde será corrido o servidor para a variável `base_folder` e também é armazenado a cache máxima na variável `cache_size`. Em seguida é iniciado o FIFO do servidor e, com recurso à função `load_index` é recuperada toda a informação em relação a documentos que estão na database do servidor.

Após criar o FIFO do servidor é iniciado um ciclo `while` que será corrido enquanto o cliente não o desligar. Dentro deste ciclo é aberto o file descriptor do servidor e é armazenado na variável `n` o pedido feito pelo cliente. Após armazenar em `n` a string passa por um ciclo `while` onde é separado cada argumento desta. Em `args[0]` é armazenada a operação desejada pelo cliente para que passe por 5 ifs onde depois os seus argumentos são enviados para a respetiva função onde será tratado o pedido do cliente. No caso de a operação pedida ser “PROCURA” e o cliente não ter especificado o número de processos necessários é enviado um 1 no lugar, sendo este o número mínimo possível.

Quando solicitado o encerramento do servidor é utilizada a função `save_index` para armazenar a base de dados e fechado o FIFO do servidor.

4. Otimização do código

Como solicitado no enunciado, foi modificada a função de pesquisa de documentos com um novo argumento de número de processos e foi testada para verificar a sua eficiência, que pode ser comprovado a partir das seguintes imagens.



```
nelson-sousa@nelson-sousa-Legion-5-15ACH6H:~/Desktop/ProjetoS0$ time ./dclient -s "ola"
[99,777]
real    0m2.587s
user    0m0.000s
sys     0m0.002s
nelson-sousa@nelson-sousa-Legion-5-15ACH6H:~/Desktop/ProjetoS0$ time ./dclient -s "ola" 5
[99,777]
real    0m0.489s
user    0m0.001s
sys     0m0.000s
```

Na primeira iteração, foi utilizado apenas um processo, o que resultou numa demora de 2,587 segundos. Por outro lado, ao recorrer a cinco processos, o tempo necessário para obter a resposta foi reduzido para 0,489 segundos, diminuindo significativamente o tempo de execução e aumentando a eficiência da operação.

5. Conclusão

Em suma, consideramos que o projeto desenvolvido satisfaz os objetivos definidos no início para a implementação deste orquestrador de tarefas.

Como possível desenvolvimento futuro, poderão ser exploradas abordagens para otimizar ainda mais a execução das tarefas, assim como a criação de novos algoritmos de escalonamento.